WEEK-3 PRACTICAL

Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

CODE:

navaneeth@Navaneeth: ~

GNU nano 8.7.1

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started\n");
    printf("Parent PID  : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());
    printf("State       : Running\n\n");

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process created\n");
        printf("Child PID   : %d\n", getpid());
        printf("Child PPID  : %d\n", getppid());
        printf("State       : Running\n");

        sleep(3);

        printf("Child PID %d\n", getpid());
        printf("State       : Terminated\n");

        exit(0);
    }
    else
    {
        printf("\nParent process\n");
        printf("Parent PID  : %d\n", getpid());
        printf("Child PID   : %d\n", pid);
        printf("State       : Waiting\n");

        wait(NULL);

        printf("Child process completed\n");
        printf("Parent State: Running\n");
        printf("Parent State: Terminated\n");
    }

    return 0;
}
```

OUTPUT:

```
navaneeth@Navaneeth:~$ gcc ParentChild.c -o ParentChild
navaneeth@Navaneeth:~$ ./ParentChild
Parent process started
Parent PID  : 597
Parent PPID : 322
State       : Running


Parent process
Parent PID  : 597
Child PID   : 598
State       : Waiting
Child process created
Child PID   : 598
Child PPID  : 597
State       : Running
Child PID 598
State       : Terminated
Child process completed
Parent State: Running
Parent State: Terminated
navaneeth@Navaneeth:~$
```

REPORT: The program successfully creates a child process using fork(). The PID and PPID of the parent and child processes are displayed, and the process states are observed during execution.

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations

CODE:

PS -EF:

```
navaneeth@Navaneeth:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 05:01 ?        00:00:00 /sbin/init
root           2       1  0 05:01 hvc0     00:00:00 /init
root           8       2  0 05:01 hvc0     00:00:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd
root          48       1  0 05:01 ?        00:00:00 /usr/lib/systemd/systemd-journald
systemd+      80       1  0 05:01 ?        00:00:00 /usr/lib/systemd/systemd-resolved
root          86       1  0 05:01 ?        00:00:00 /usr/lib/systemd/systemd-udevd
root         106       1  0 05:01 ?        00:00:00 /bin/sh /usr/lib/systemd/scripts/chronyd-starter.sh -n -F 1
root         107       1  0 05:01 ?        00:00:00 /usr/sbin/cron -f -P
message+     108       1  0 05:01 ?        00:00:00 @dbus-daemon --system --address=systemd: --nofork --nopidfile
root         113       1  0 05:01 ?        00:00:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-tr
root         149       1  0 05:01 ?        00:00:00 /usr/lib/systemd/systemd-logind
syslog       180       1  0 05:01 ?        00:00:00 /usr/sbin/rsyslogd -n -iNONE
root         194       1  0 05:01 tty1     00:00:00 /usr/sbin/agetty --noreset --noclear --issue-file=/etc/issue:/
_chrony      204     106  0 05:01 ?        00:00:00 /usr/sbin/chronyd -n -F 1 -x
root         206       1  0 05:01 ?        00:00:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upg
_chrony      228     204  0 05:01 ?        00:00:00 /usr/sbin/chronyd -n -F 1 -x
root         320       2  0 05:01 ?        00:00:00 /init
root         321     320  0 05:01 ?        00:00:00 /init
navanee+     322     321  0 05:01 pts/0    00:00:00 -bash
root         323       2  0 05:01 ?        00:00:00 login -- navaneeth
navanee+     365       1  0 05:01 ?        00:00:00 /usr/lib/systemd/systemd --user
navanee+     367     365  0 05:01 ?        00:00:00 (sd-pam)
navanee+     393     323  0 05:01 pts/1    00:00:00 -bash
navanee+     637     322  0 05:18 pts/0    00:00:00 ps -ef
navaneeth@Navaneeth:~$
```

PS:

```
navaneeth@Navaneeth:~$ ps -o pid,ppid,state,cmd
    PID    PPID S CMD
    322     321 S -bash
    649     322 R ps -o pid,ppid,state,cmd
navaneeth@Navaneeth:~$
```

TOP:

```
navaneeth@Navaneeth:~$ top
top - 05:30:35 up 28 min,  1 user,  load average: 0.00, 0.01, 0.00
Tasks:  27 total,   1 running,  26 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :   7762.6 total,   7104.1 free,    539.1 used,    270.0 buff/cache
MiB Swap:   2048.0 total,   2048.0 free,      0.0 used.   7223.6 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   24184  15296  11512 S   0.0   0.2   0:00.53 systemd
      2 root      20   0    3180   2204   2072 S   0.0   0.0   0:00.01 init-systemd(Ub
      8 root      20   0    3180   2044   1940 S   0.0   0.0   0:00.00 init
     48 root      19  -1   50400  16708  15428 S   0.0   0.2   0:00.15 systemd-journal
     80 systemd+  20   0   22540  14608  11984 S   0.0   0.2   0:00.06 systemd-resolve
     86 root      20   0   35244  12344   9252 S   0.0   0.2   0:00.57 systemd-udevd
    106 root      20   0    2888   1936   1808 S   0.0   0.0   0:00.04 chronyd-starter
    107 root      20   0    4472   3104   2860 S   0.0   0.0   0:00.01 cron
    108 message+  20   0    8820   5312   4644 S   0.0   0.1   0:00.05 dbus-daemon
    113 root      20   0   44092  29808  14672 S   0.0   0.4   0:00.21 networkd-dispat
    149 root      20   0   18436   9448   8200 S   0.0   0.1   0:00.06 systemd-logind
    180 syslog    20   0  220548   5884   4624 S   0.0   0.1   0:00.06 rsyslogd
    194 root      20   0    5492   2748   2564 S   0.0   0.0   0:00.00 agetty
    204 _chrony   20   0   24552  12096   7228 S   0.0   0.2   0:00.09 chronyd
    206 root      20   0  123456  32616  18008 S   0.0   0.4   0:00.16 unattended-upgr
    228 _chrony   20   0   12092   2464   1820 S   0.0   0.0   0:00.00 chronyd
    320 root      20   0    3184   1108    980 S   0.0   0.0   0:00.00 SessionLeader
    321 root      20   0    3200   1248   1108 S   0.0   0.0   0:00.09 Relay(322)
    322 navanee+  20   0    6268   5528   3856 S   0.0   0.1   0:00.09 bash
    323 root      20   0    8652   5524   4676 S   0.0   0.1   0:00.00 login
    365 navanee+  20   0   22752  13328  10868 S   0.0   0.2   0:00.08 systemd
    367 navanee+  20   0   22912   4092   2088 S   0.0   0.1   0:00.00 (sd-pam)
    393 navanee+  20   0    6136   5408   3860 S   0.0   0.1   0:00.01 bash
    699 root      20   0    3184   1116    980 S   0.0   0.0   0:00.00 SessionLeader
    700 root      20   0    3200   1256   1108 S   0.0   0.0   0:00.01 Relay(707)
    707 navanee+  20   0    6268   5532   3856 S   0.0   0.1   0:00.06 bash
    733 navanee+  20   0    8284   6056   3900 R   0.0   0.1   0:00.01 top
```

REPORT:This is understanding process creation and process state transitions in Linux. The fork() system call was used to create a child process, while ps, top, and /proc were used to monitor process information and states. The PID and PPID helped identify the relationship between the parent and child processes.